



Development of an Automated Parametric Posture Solver for Complex Closed Kinematic Chains

Author:

Benjamin B BOLDUC

Tutors:

José O. NEVES

Francesco SANFEDINO

Daniel ALAZARD

July 1, 2026

Contents

1. Introduction	5
2. Project Evolution	6
2.1. Review of Previous Work	6
2.2. Key Issues	6
2.3. New Objective and Shift in Approach	6
3. State of the Art	7
3.1. Commercial Solvers vs. Custom Integration	7
3.2. Graph Theory in Kinematics	8
3.3. Advanced Numerical Optimization	8
3.4. 3D Spatial Representation	9
4. Generalized Algorithmic Architecture	10
4.1. Abstract Mechanism Definition	10
4.2. Automated Loop Discovery	10
4.3. Dimensionality Detection	12
4.4. Formulation of Minimization Parameters	12
4.4.1. Planar Formulation	12
4.4.2. The Spatial Formulation	15
4.4.3. The Trade-off Between the Formulations	16
4.5. The Mathematical Engine	16
4.5.1. Initial State Formulation and Kinematic Continuity	16
4.5.2. Numerical Optimization via Levenberg-Marquardt	17
4.5.3. Metric Extraction and Post-Processing	17
5. Computational Performance and Validation	18
5.1. Baseline Verification	18
5.2. Planar Scalability	20
5.3. Spatial Validation	21
5.4. Application to the ATHENA Hexapod	22
5.4.1. Modelling the Hexapod	22
5.4.2. Resolving Overconstrained Singularities via Kinematic Equivalence	23
5.4.3. Solving the Athena Hexapod	23
6. Conclusion	25
A. Four-bar Mechanism JSON File Example	28
B. JSON File Nomenclature	30
B.1. Ternary Links	30
B.2. Prismatic Joints	30
B.3. Spherical Joints	30

Declaration of Authenticity

This assignment is entirely my own work. Quotations from literature are properly indicated with appropriate references in the text. All literature used in this piece of work is indicated in the bibliography placed at the end. I confirm that no sources have been used other than those stated.

I understand that plagiarism (copy without mentioning the reference) is a serious examination offence that may result in disciplinary action being taken.

Date: July 1, 2026

Signature: *Benjamin B Bolduc*

Abstract

In space robotics, tools like the Satellite Dynamics Toolbox (SDTlib) rely on Linear Fractional Transformations (LFTs) to model parametric uncertainties. However, closed kinematic chains introduce implicit non-rational constraints that directly prevent LFTs from being used. To address this, LFTs can be approximated, but doing so requires the computation of the exact constrained parameters. This computation is often either extremely time-consuming or outright impossible. To overcome this, this research develops a parametric numerical solver that is capable of computing the constrained posture of both planar and spatial multi-loop mechanisms. By defining the mechanism topologies into a standardized JSON architecture, the custom solver leverages graph theory to find all of the mechanism's independent kinematic loops. Then, the Levenberg-Marquardt numerical optimization algorithm is employed to minimize pre-defined parametric loop closure constraints. From this, the custom solver was finally verified and validated against a series of closed kinematic chains and ultimately ESA's Athena spacecraft's highly complex hexapod platform. In the case of the hexapod, the solver achieved a high precision accuracy of 10^{-16} m. By successfully solving the posture of closed kinematic chains in a parametric manner, the developed solver effectively bridges the gap between physical multi-body systems and the approximation of LFTs.

Keywords: Closed kinematic chains, space robotics, graph theory, Levenberg-Marquardt optimization, inverse kinematics, parametric posture solver, Linear Fractional Transformations (LFT).

1. Introduction

Recent collaborations between NASA and the European Space Agency (ESA) have highlighted a critical need for the development of preliminary design tools capable of guaranteeing robust spacecraft control performance during early design phases [1]. ISAE-SUPAERO has responded to this by developing the Satellite Dynamics Toolbox library (SDTlib), a MATLAB/Simulink library designed to model complex, multi-body, flexible structures [2].

To manage the uncertainties inherent in preliminary spacecraft design, SDTlib uses the Linear Fractional Transformation (LFT) paradigm to parameterise its mathematical models [3]. However, in closed kinematic chains, the total number of degrees of freedom is less than the sum of the degrees of freedom of each individual body. Because SDTlib assigns one parameter to every degree of freedom, this restriction creates dependent parameters that must be suppressed using implicit, non-rational constraints. As a result, the constrained parameters cannot be directly represented as LFTs. This is resolved by approximating the LFTs with rational functions, which first requires computing the exact constrained parameters.

Manually deriving explicit expressions for this computation is mathematically impossible. While the implicit constraint equations can be formulated, doing so manually is highly time-consuming and error-prone. Therefore, the primary objective of this research is to develop a custom parametric solver that automatically formulates these implicit constraint equations from a set of free parameter values, allowing the constrained parameters to be determined without manual user intervention. To validate this tool's applicability, the custom solver will be tasked with evaluating the posture of the ESA Athena spacecraft's secondary mirror mechanism, the hexapod [4]. By evaluating this complex mechanism, the research will prove that the custom solver can be used in the context of building rational LFT approximations for closed kinematic chains in SDTlib.

2. Project Evolution

2.1. Review of Previous Work

During this project's first semester, a lot of importance was attributed to solving the constraint parameters of closed-loop kinematic chains. This was achieved using an iterative approach where foundational kinematic chains such as the planar four-bar linkage were explored. A review and trade study of state of the art numerical methods concluded in the development of a custom Newton-Raphson numerical solver. In the early stages of development of this solver, a key innovation was made: the use of a global angle representation rather than using a local reference frame. This approach significantly decreased the non-linearity of the constraint parameters by removing unnecessary constraint parameters from non-linear terms. It proved to be highly effective, validating against known analytical solutions up to tolerances of 10^{-4} . Other metrics such as the solver's iteration count were also considered.

2.2. Key Issues

While the solver's first iteration worked well for foundational linkages like the four-bar mechanism, several limitations surfaced when testing increasingly complex systems. For instance, the workflow relied on simplifying assumptions such as assuming each link only had exactly two connection points and forcing a specific traversal direction to solve the chain. Additionally, the workflow required the user to provide an initial guess for the system's Jacobian. Given the abstract nature of Jacobian matrices, this introduced different possible negative outcomes for the solver. Most notably non-convergence, high iteration counts, or singularities. Seeing the solver behave as such for simple closed kinematic chains like the four-bar linkages meant this behaviour would only deteriorate with increasingly complex systems.

2.3. New Objective and Shift in Approach

To overcome the limitations of the preliminary algorithm and fulfill the ultimate goal of processing the Athena hexapod, a fundamental pivot in the core architecture was required. The objective shifted from writing a custom numerical root-finding algorithm to developing a generalized, parametric loop-closure equation generator. This pivot allowed the project to refocus on the autonomous formulation of kinematic constraints by leveraging the stability of proven, robust numerical solvers, such as Levenberg-Marquardt's and Powell's dogleg methods [5]. To achieve this, a graph-theory approach was implemented. The new custom solver is fed a JSON file describing a mechanism's bodies and joints, it then autonomously identifies the independent kinematic loops, and finally generates the exact residual equations without any manual *a priori* hardcoding.

3. State of the Art

This section aims to review the literature relevant to the development of a numerical solver used to parametrically determine the posture of closed kinematic chains. To justify the decisions made in the development of this solver, this section first evaluates the limitations of existing, industry-standard commercial tools. Then, it explores modern approaches used in the following domains: the application of graph theory for autonomous mechanism topology discovery, the necessity of advanced numerical optimization algorithms for robust convergence, and addressing singularities in 3D spatial representations. By setting this foundation, it demonstrates how the challenges of developing the custom solver were addressed using state of the art solutions.

3.1. Commercial Solvers vs. Custom Integration

The industry standard for evaluating multibody dynamics relies heavily on commercial simulation packages such as *Simscape Multibody* or *Adams* due to their robustness and ability to tackle highly complex problems. These introduce limitations from a user perspective, however, in their operational philosophy. Block-based environments, while inherently user friendly, act as black boxes. In being user-friendly, they completely obscure crucial information from the user such as constraint equations, Jacobian matrices, and solver algorithms. Furthermore, these commercial solvers typically require significant computational power, making them inefficient in contexts requiring a large number of iterative numerical optimization evaluations.

As discussed by Wittenburg in *Dynamics of Multibody Systems*, relying on closed software severely limits an engineer's ability to analyze new mechanisms [6, pp. 89-90]. He then argues that a generalized, mathematically explicit formalism is required [6, pp. 89-90]. For this to be the case, adapting a solver to different mechanical systems should only require the modification of abstract input data such as bodies, joints, and constraints [6, p. 126]. In his text he also expresses the needs of open-source solver architectures. Unlike commercial solvers, like Simscape, open-source code can be easily integrated into existing architectures based on the use-case. This argument is also supported by Schiehlen where it is concluded that transparent, explicitly derived kinematics provide the mathematical flexibility required to solve any possible system structure [7, p. 163].

To address these limitations, modern multi-body tools have taken a shift toward code-based engines rather than their block-diagram counterparts. For instance, the open-source *Project Chrono* is capable of handling large multi-physics systems due to it relying on a C++ and text-based architecture [8, pp. 1-3]. Text-based architectures can also be observed in works led by Zachary Manchester and his team in developing physics engines such as *Dojo* [9, pp. 1-3].

However, while existing open-source frameworks like *Project Chrono* and *Dojo* make use of the desired architecture for this research, they are at their core designed as full-scale dynamics and physics simulators. This research aims to efficiently and parametrically solve the kinematic posture of closed kinematic chains while *Project Chrono* and *Dojo* were built to solve complex mechanics and dynamics problems. As such, using these extensive engines to strictly meet the objective of this research introduces a steep learning curve and overall software bloat. Therefore, to maintain control over the software and to ensure that it remains lightweight and only addresses this research's objectives, a custom mathematical engine was developed. Using a standardized text-based approach as argued and proven allows for the explicit, automated derivation of kinematic relationships and constraints. It ensures the mathematical engine remains flexible, computation-

ally lightweight, and perfectly tailored to solve highly constrained systems, such as the Athena hexapod, without being reliant on proprietary, computationally demanding, graphical interfaces.

3.2. Graph Theory in Kinematics

Historically, the posture of closed kinematic chains has been solved through the manual derivation of the loop-closure equations. Evidently, as these kinematic chains increase in complexity and introduce multiple loops, the equations become more difficult and convoluted to derive. This method is not only prone to human error, but it is incompatible with the primary objective of this research—developing a parametric closed kinematic chain posture solver. To address this limitation, modern approaches have introduced the use of graph theory. This has been demonstrated previously in Tsai’s work where a mechanism’s bodies act as vertices and joints as edges to finally yield a full mathematical graph representation [10, pp. 9-15].

While mathematical graphs are a robust way of representing mechanisms, they introduce the challenge of identifying said mechanism’s independent kinematic loops. To achieve this, Wittenburg explains that the adjacency matrix obtained using graph theory can be fed into an algorithm capable of generating a network connecting all bodies without forming any closed loops [6, p. 129]. From this, any joint not included in this network is classified as a chord, and reintroducing a single chord back into the tree defines exactly one independent kinematic loop, known as a fundamental cycle [6, p. 129]. This is further promoted by Müller who states that this method guarantees that the formulated loop-closure constraints are strictly independent and that every single one of a mechanism’s independent kinematic loops are found [11]. Computationally, this research addresses this challenge by taking a text-based mechanism definition and feeding it through MATLAB’s robust native graph theory function, `cyclebasis`. By leveraging these optimized algorithms, the solver is allowed to autonomously and robustly discover a mechanism’s independent kinematic loops.

3.3. Advanced Numerical Optimization

Common numerical optimization solvers often make use of the Newton-Raphson method. While effective for simple nonlinear systems of equations and rapid implementation, the Newton-Raphson method requires taking the inverse of the system’s Jacobian matrix at every iteration. As both Nocedal and Wright detail, when a system approaches a kinematic singularity, or if the initial guess is far from the true root, the Jacobian becomes ill-conditioned, causing the computed step size to trend toward infinity and resulting in numerical instability [12, pp. 262-266]. Other numerical optimization methods such as Broyden’s method attempt to address this by approximating the Jacobian rather than computing it, but it still suffers from the possibility of singularities and the failure to converge.

As such, to guarantee solver convergence, a more robust optimization algorithm is required. The solution lies in the Levenberg-Marquardt algorithm. This numerical optimization algorithm resolves the singularity issue through the implementation of a damped least-squares approach. Sugihara proves that this approach mathematically prevents the matrix inversion from diverging, leading to improved numerical stability and guaranteed convergence for near singular configurations where Newton-Raphson and Broyden’s methods would otherwise fail [5]. This algorithm can be used inside MATLAB’s `fsolve` function.

3.4. 3D Spatial Representation

3D rotational kinematics are a key subject of this research. Often, these are computed using sequential Euler angles, an approach extremely intuitive for human interpretation. However, as discussed by Diebel, this approach suffers from a mathematical phenomenon known as gimbal lock. This occurs when two rotational axes perfectly align, taking away the rank from the rotation matrix's Jacobian, causing numerical algorithms to fail [13, p. 13].

This is addressed by instead using the principal axis-angle method. As outlined by Brezov, instead of three sequential rotations, this method evaluates a single rotation about a generalized unit vector, completely circumventing gimbal lock [14]. By integrating this method, the architecture developed in this research reduces the rotational residual of spatial mechanisms from a 3×3 rotation matrix down to a more manageable, 3×1 orientation error vector. Furthermore, this formulation allows for the direct representation of rotations based strictly on user input, as the user only needs to specify a single rotation angle about a known axis.

4. Generalized Algorithmic Architecture

The primary objective of this research is to develop a numerical solver capable of parametrically solving the posture of planar and spatial closed kinematic chains. Such architecture is highly complex and has to be separated into a series of different challenges to overcome, all highlighted in this section. These challenges include parsing an abstract mechanism definition, autonomously mapping kinematic chains using graph theory, evaluating dimensionality, formulating loop-closure residuals, and driving those residuals to zero using a robust numerical optimization algorithm.

4.1. Abstract Mechanism Definition

As discussed in Section 3.1, commercial solvers such as Simulink’s Simscape Multibody provide highly robust simulation environments. Their limitation, however, lies in how the user builds mechanisms using the graphical block-diagram interface. While extremely versatile for rapid prototyping, this method acts as a black box, completely obscuring the constraint equations formulation from the user. Since the primary objective of this research is to develop a parametric numerical solver for the posture of closed kinematic chains, a transparent architecture was required. By taking inspiration from other modern approaches such as *Project Chrono* and Zachary Manchester’s work, the chosen architecture shifts from a graphical block-diagram interface to text-based definitions of rigid bodies and constraints. This allowed for mechanisms to then be defined completely independently from the mathematical engine using an abstract standardized JSON data structure.

This abstract mechanism definition was built in a way that divides any mechanism into two categories: rigid bodies and kinematic joints.

- **Rigid bodies:** Defined by an identifier and an array of physical connection points. Each point contains 3D Cartesian coordinates ($[X, Y, Z]$) defining the body’s geometry.
- **Kinematic joints:** Defined by an identifier, the pair of bodies it connects, the joint type (revolute or prismatic), and its unit axis of motion ($[X, Y, Z]$).

This architecture relies on a strict nomenclature to ensure the topological continuity of a mechanism. To enforce this nomenclature, the `label` of a joint must perfectly match the `id` of the connection point on its corresponding body. When initialized, the solver parses this standardized JSON architecture and converts it into a data structure readable by MATLAB, setting the stage for automated graph traversal. An example of this JSON architecture applied on the four-bar mechanism can be found in Appendix A.

To ensure the parser remained universally applicable, all multi-DOF connections were reduced to their fundamental 1-DOF primitives using kinematic equivalence. For example, instead of programming a unique mathematical object for a 3-DOF spherical joint, the JSON defines it by stacking three perpendicular 1-DOF revolute joints at identical coordinates. Detailed JSON formatting examples for special cases such as ternary linkages, prismatic joints, and spherical joints can be found in Appendix B.

4.2. Automated Loop Discovery

Once the mechanism is successfully abstracted into a JSON data structure, the mathematical engine must autonomously identify the independent kinematic constraints. This is traditionally done by manually deriving the loop-closure equations. However, this cannot be the case for a parametric architecture such as the one expected from this research. To automate this derivation

process, the principles of graph theory were employed. This is done by having the solver's parser translate the JSON architecture into a graph where the mechanism's bodies act as nodes and the joints act as the edges connecting these nodes. In MATLAB, this is represented by populating an $N \times N$ adjacency matrix, where N represents the total number of rigid bodies in the system. To populate this matrix, the following logic is used:

- If a body i and a body j share a physical joint, the corresponding matrix entry $A_{i,j}$ is assigned a value of 1.
- If a body i and a body j do not share a physical joint, the corresponding matrix entry $A_{i,j}$ is assigned a value of 0.

This is shown in Figure 1 using the four-bar mechanism as an example.

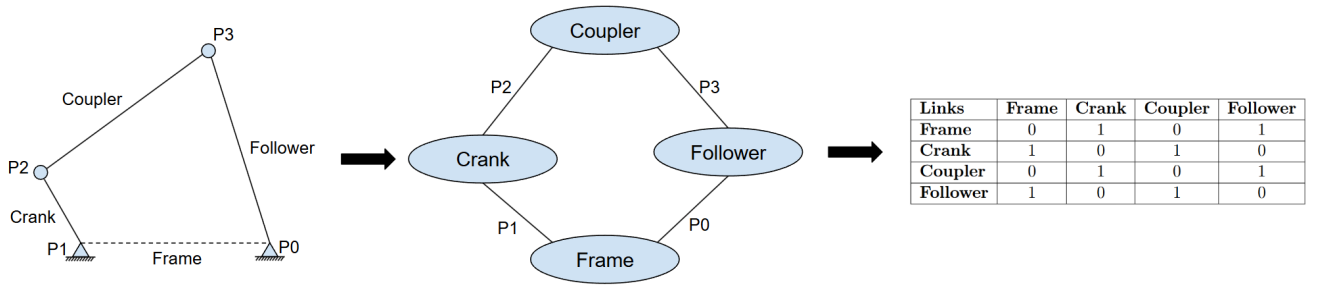


Figure 1: Graph theory workflow example on the four-bar mechanism.

By converting this adjacency matrix into a MATLAB `graph` object, the solver can leverage robust built-in tools. Specifically, the `cyclebasis` function is used to compute a fundamental cycle basis of an input graph, outputting the specific nodes that belong to each fundamental cycle [15]. In the context of this research, these extracted cycles correspond directly to each independent closed kinematic loop within the broader mechanism. An example taken from MATLAB's manual on the `cyclebasis` function is shown in Figure 2.

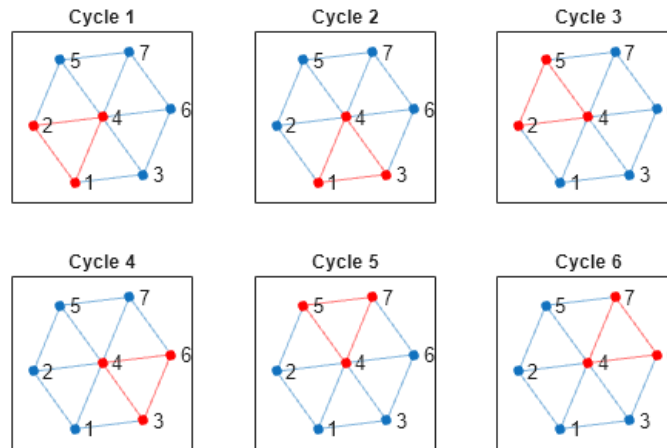


Figure 2: Independent loop example [15].

By automating this process, the solver is guaranteed to function for any mechanism, regardless of its complexity, without *a priori* human intervention. Furthermore, this method not only discovers the independent closed kinematic loops, but it yields the specific order of bodies and joints within

them. This sequential mapping provides the exact foundation required to dynamically construct the loop-closure equations for posture determination.

4.3. Dimensionality Detection

Once the independent kinematic loops have been successfully extracted from the graph, the appropriate mathematical solver must be selected. While a 3D solver is mathematically capable of solving any mechanism, applying it to a planar system introduces unnecessary computational overhead. To optimize performance, a dimensionality detection algorithm was implemented to evaluate whether the mechanism can be safely solved using the faster and less computationally demanding 2D planar equations.

For a mechanism to be considered planar, two geometric conditions must be met:

- **Planar bodies:** All body vectors must lie perfectly flat in the $X - Y$ plane. In other words the Z -component of the body vectors must be zero.
- **Planar joints:** All joints axes must align with planar degrees of freedom. For the mechanism to remain 2D, the axis unit vector, $\mathbf{u}$, must either be normal to the $X - Y$ plane (revolute joint $|\mathbf{u} \cdot \mathbf{k}| = 1$), or lie entirely within the $X - Y$ plane (prismatic joint, $|\mathbf{u} \cdot \mathbf{k}| = 0$).

These conditions are verified with the algorithm by iterating through the local joint axes and body vectors. If any depth is observed in the Z -axis or if any joint axis is skewed, the algorithm flags the mechanism as 3D. By automatically detecting the mechanism's dimensionality using the information provided by the graph parser, the solver ensures that the optimal amount of computational resources are used and that the mechanism is solved using the correct solver branch.

It is important to note that this geometric detection logic assumes a strictly planar configuration. A multi-planar mechanism exhibits 2D kinematics when its bodies occupy three-dimensional space but all of its joint axes remain perfectly parallel. Under the current logic, this would be flagged as 3D, relying on the spatial formulation to solve the posture.

4.4. Formulation of Minimization Parameters

Once a mechanism's JSON topology is successfully parsed and its dimensionality is determined, the objective functions that the solver will aim to minimize need to be obtained. This is achieved by enforcing the loop-closure constraints and calculating the resulting residuals. To do this, the solver traverses the extracted kinematic loops obtained in Section 4.2 and applies the transformations to the mechanism's bodies based on the numerical solver's current estimated solution. Fundamentally, a closed kinematic chain must achieve loop closure, meaning that the cumulative sum of all body vectors must equal zero. Depending on the dimensionality obtained in Section 4.3, the solver proceeds either with planar (2D) or spatial (3D) evaluation.

4.4.1. Planar Formulation

When detected as two dimensional, the mechanism is evaluated using a global-angle approach rather than its traditional local-angle counterpart. Typically, a revolute joint's angular position is defined with respect to a local reference frame. In the case of the four-bar mechanism, the diagram in Figure 3 can be drawn and the loop-closure equations in Eq. (1) can be derived.

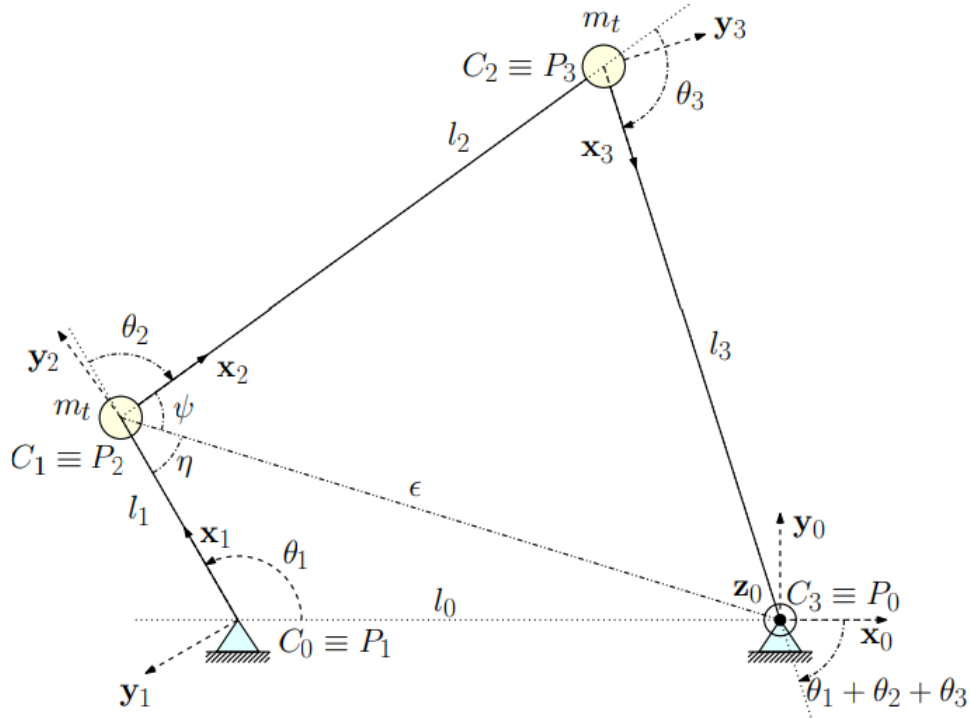


Figure 3: Four-bar mechanism with local-angle representation.

$$\begin{cases} l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2) + l_3 \cos(\theta_1 + \theta_2 + \theta_3) &= l_0 \\ l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2) + l_3 \sin(\theta_1 + \theta_2 + \theta_3) &= 0 \end{cases} \quad (1)$$

It can be shown that this approach relies on the cumulative sum of the joint angles inside nonlinear trigonometric terms creating highly coupled nonlinear constraint equations. By instead tracking the absolute orientation of each joint (θ') with respect to a fixed global reference frame, as shown in Figure 4, the target variables become decoupled.

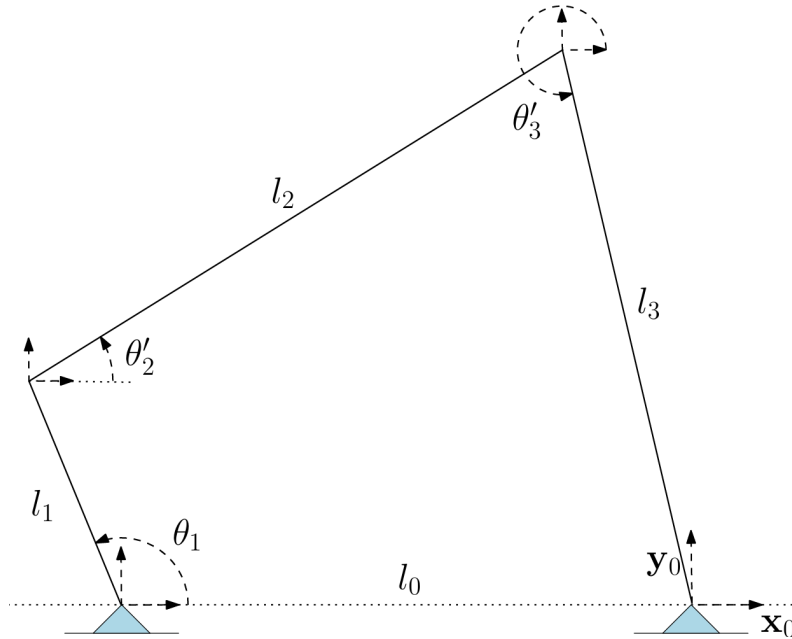


Figure 4: Four-bar mechanism with global-angle representation.

Rederiving the loop-closure equations using the global-angle formulation then yields Eq. (2).

$$\begin{cases} l_1 \cos \theta'_1 + l_2 \cos(\theta'_2) + l_3 \cos(\theta'_3) = l_0 \\ l_1 \sin \theta'_1 + l_2 \sin(\theta'_2) + l_3 \sin(\theta'_3) = 0 \end{cases} \quad (2)$$

By shifting the ground frame vector, l_0 , to the left side of the equation, a clear pattern can be observed, allowing the set of equations to be simplified into the generalized formulation seen in Eq. (3).

$$\begin{cases} \sum_{i=1}^n l_i \cos \theta'_i = 0 & \text{(x-closure)} \\ \sum_{i=1}^n l_i \sin \theta'_i = 0 & \text{(y-closure)} \end{cases} \quad (3)$$

This is approached differently, however, when the solver detects a prismatic joint. Where a revolute joint uses the global angle, θ'_i , as the minimization variable, a prismatic joint uses the linear actuator extension, d_i , as the minimization variable. In this case, the global angle remains constant and constrained to the orientation of the prismatic joint's sliding track. Modifying Eq. (3) to include prismatic displacement yields the generalized 2D loop-closure formulation in Eq. (4).

$$\begin{cases} \sum_{i=1}^n (l_i + d_i) \cos \theta'_i = 0 & \text{(x-closure)} \\ \sum_{i=1}^n (l_i + d_i) \sin \theta'_i = 0 & \text{(y-closure)} \end{cases} \quad (4)$$

In the case of revolute joints $d_i = 0$, so the formulation simplifies back to solving exclusively for θ'_i . In the case of prismatic joints, θ'_i is a known constant, so the solver isolates and solves for d_i .

Naturally, this formulation yields a set of two equations for n -unknowns, making the system under-constrained. As such, additional constraints need to be enforced in order for the system to be fully solvable. These constraints are:

- **Ground orientation constraint:** The first body is physically fixed to the environment. This is done by arbitrarily setting this body's global angle to $\theta'_k = \pi$, locking the mechanism in global space.
- **Input joint constraint:** The geometric relation between the global and local angles of any two connected bodies is defined as:

$$\theta'_{k+1} - \theta'_k = \theta_k \quad (5)$$

This constraint is enforced for every known input angle to ensure that the system maintains an equal number of equations to unknowns.

By combining and parametrically enforcing these constraints, the solver can efficiently minimize the desired parameters in two dimensions. To illustrate this workflow, the full residual vector for the four-bar mechanism from Figure 4 is shown in Eq. (6) with θ_1 as the input angle:

$$\mathbf{E} = \begin{bmatrix} l_0 \cos \theta'_0 + l_1 \cos \theta'_1 + l_2 \cos \theta'_2 + l_3 \cos \theta'_3 \\ l_0 \sin \theta'_0 + l_1 \sin \theta'_1 + l_2 \sin \theta'_2 + l_3 \sin \theta'_3 \\ \theta'_0 - \pi \\ \theta'_2 - \theta'_1 - \theta_1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (6)$$

This system of 4 equations and 4 unknowns ($\theta'_1, \theta'_2, \theta'_3, \theta'_4$) would then be passed to the next step of the solver, the numerical optimization engine.

4.4.2. The Spatial Formulation

When the mechanism is detected as three dimensional, the solver switches to the spatial formulation, implementing the principal axis-principal angle method. In 3D space, a shift from 3 degrees of freedom to 6 degrees of freedom can be observed. These consist of 3 translation degrees of freedom ($[X, Y, Z]$) and 3 rotation degrees of freedom ($[\phi, \theta, \psi]$). Consequently, closing a three dimensional kinematic chain imposes the following constraints on the final body in the kinematic loop:

- **Position constraint:** The end of the chain must return to the starting point ($[\Delta X, \Delta Y, \Delta Z]^T = 0$).
- **Rotation constraint:** The final body's orientation must match the initial body's orientation ($R_{total} = I_3$).

To evaluate the first constraint, the position closure can be computed through the cumulative sum of the rotated link vectors, which must equal zero per Eq. (7):

$$r_{pos} = \sum_{i=1}^n R_{1:i} v_i = 0 \quad (7)$$

Where v_i is the vector of body i , and $R_{1:i}$ is the cumulative rotation matrix up to that body. This rotation matrix is calculated by successively multiplying the relative rotation matrices, $R(\hat{a}_k, \theta_k)$, generated at each joint around a principal unit axis, $\hat{a}_k$, by and a local angle, θ_k :

$$R_{1:i} = \prod_{k=1}^i R(\hat{a}_k, \theta_k) \quad (8)$$

This can then be used to evaluate the second constraint which dictates that a complete rotation around a closed kinematic chain must yield the 3×3 identity matrix (I_3):

$$R_{total} = \prod_{k=1}^n R(\hat{a}_k, \theta_k) = I_3 \quad (9)$$

To translate this into information the solver can use to minimize the desired parameters, R_{total} can be converted to $r_{orientation}$ through the principal axis-angle representation shown in Eq. (10) (known as `rotm2axang` in MATLAB).

$$r_{orientation} = \theta \cdot \hat{k} \quad (10)$$

Where θ is the equivalent rotation angle about a unit axis $\hat{k}$. This reduces the number of minimization parameters from 9 ($R_{total, [3 \times 3]}$) down to 3 ($r_{orientation, [1 \times 3]}$), yielding the residual vector for the second constraint.

Concatenating the residuals from the two constraints yields the six minimization parameters required to compute the posture of a 3D closed kinematic chain.

$$r = \begin{bmatrix} r_{pos} \\ r_{ori} \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \end{bmatrix} \quad (11)$$

It should be noted that the solver strictly minimizes these geometric residuals. The joint axis vectors are not directly solved because their global orientations are determined by the cumulative rotation matrices.

4.4.3. The Trade-off Between the Formulations

A key observation can be made when considering the planar and spatial mathematical formulations: the spatial formulation is universally applicable, even in two dimensions. By making use of successive matrix multiplication, it can evaluate any kinematic topology without the need of careful constraint definitions. However, continuously computing 3×3 rotation matrices, and extracting the principal axis-angle representation at every solver iteration can be extremely computationally demanding.

Conversely, the planar formulation was intentionally designed and optimized strictly for computational efficiency. While the initial setup of the 2D approach is more complex, completely eliminating the need for unnecessary computations and decoupling the constraint parameters from within trigonometric functions was a calculated trade-off. The computational requirement is not evident in one-off posture computations, however, in simulations requiring hundreds or thousands of mechanism posture computations, the efficiency of the 2D branch becomes noticeable. This in turn justifies the dimensionality detection algorithm discussed in Section 4.3, ensuring that the computationally demanding 3D formulation is exclusively used when necessary.

4.5. The Mathematical Engine

Having defined the minimization parameters for both the planar and spatial formulations, a robust numerical optimization algorithm to drive the residual vectors to zero is required. The objective of this algorithm is to iteratively adjust the desired constraint parameters until the loop-closure constraints derived in Section 4.4 are satisfied.

4.5.1. Initial State Formulation and Kinematic Continuity

One of the challenges with handling closed kinematic chains is that multiple solutions are often possible for a given set of inputs. For example, this can happen with the four-bar linkage when both an "open-above" and "crossed-above" configuration can be observed for the same input angle as shown in Figure 5.

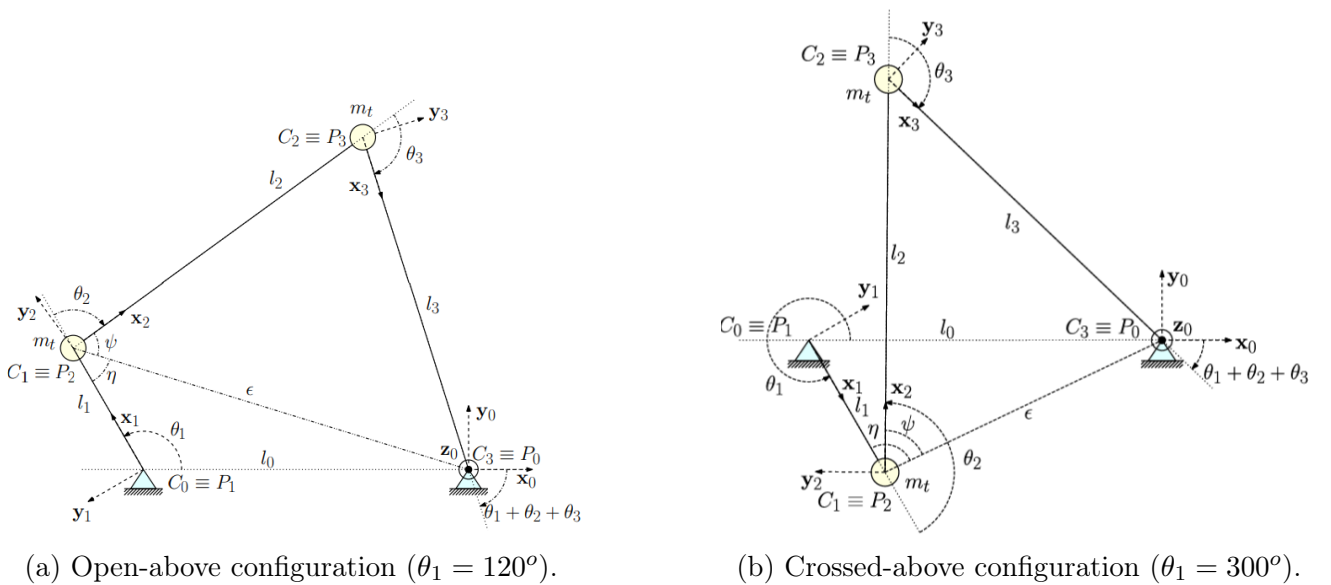


Figure 5: Example of a four-bar mechanism's configurations.

To avoid the solver from jumping between configuration during simulations, the engine uses the solution from one posture as the initial guess for the next posture. As such, the solver is biased toward the same configuration for the next solution, preventing kinematic discontinuities in the visualization.

4.5.2. Numerical Optimization via Levenberg-Marquardt

To execute the minimization, the residual vectors and the user's initial guesses are fed into MATLAB's `fsolve` function. As discussed in Section 3, standard Newton-Raphson approaches are prone to instability and non-convergence when evaluating highly non-linear systems of equations [12]. Because of this, the Levenberg-Marquardt algorithm was selected for the solver.

This algorithm provides a highly robust least-squares approach, dynamically interpolating between the Gauss-Newton algorithm and the method of gradient descent [5]. To ensure the solver is robust and the correct posture is found, a strict tolerance of 10^{-14} was implemented.

4.5.3. Metric Extraction and Post-Processing

Once the numerical solver terminates and has successfully converged on a posture, the optimization algorithm outputs an array of residuals. Because this array cannot be intuitively interpreted by a human, it must be manipulated to provide the user with metrics capable of highlighting solver performance. For both the planar and spatial branches, the engine parses through the residual vector to calculate the maximum displacement between any two connected bodies (in meters) and the maximum angular error (in degrees).

5. Computational Performance and Validation

This section details the verification and validation of the custom numerical solver developed in this research. To investigate both the mathematical precision and the topological capabilities of the solver, a series of kinematic benchmarks were performed. First, the solver's baseline accuracy was investigated through the comparison of the solver's results against known analytical solutions and commercial solutions like Simulink. Then, the solver's capabilities were tested on planar linkages containing multiple kinematic loops. Following this, the solver was put to the test in validating three dimensional topologies. This finally culminated to applying the solver to ESA's Athena spacecraft's hexapod platform, proving the tool's ability to compute a highly complex closed kinematic chains' constrained parameters.

5.1. Baseline Verification

To ensure that the implementation of the graph theory approach and automated loop discovery outlined in Section 4 worked as intended, the numerical solver was first verified and validated using the four-bar mechanism. The goal with this was to prove that the parametrically derived loop closure equations yielded identical results to those from known analytical derivations and commercial solvers like Simulink.

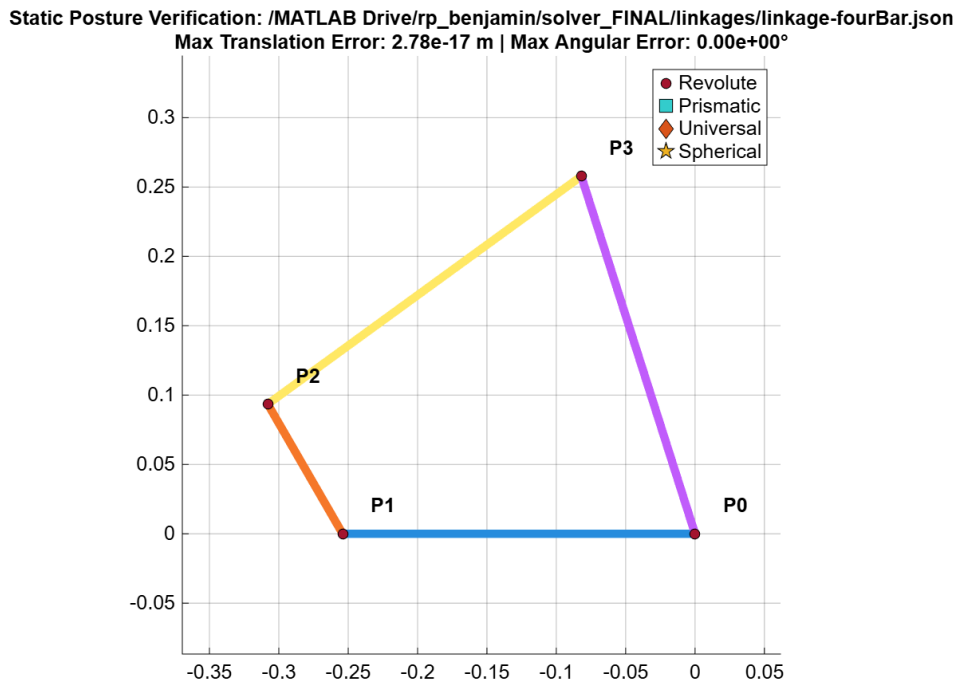


Figure 6: Static Posture verification of the Four-bar linkage.

As shown in the static posture verification in Figure 6, the solver successfully converged. A maximum translational residual of 2.78×10^{-17} m can be observed, highlighting the robustness of the Levenberg-Marquardt algorithm. The specific joint variables calculated by the autonomous solver to achieve this closed loop posture are given report-style when running the solver. This is detailed in Table 1.

Table 1: Numerical solver output for the four-bar mechanism (input: $P1 = 120^\circ$).

Joint	Link Order	Solved Angle [deg]
P1	Frame $\rightarrow$ Crank	120.00
P0	Frame $\rightarrow$ Lever	-72.36
P2	Crank $\rightarrow$ Follower	-83.98
P3	Follower $\rightarrow$ Lever	-108.38

To validate the solver's stability across the mechanism's full range of motion, a full 360° sweep of the input joint ($P1$) was simulated. This sweep was also performed using known analytical equations and a Simulink block diagram. Using the analytical results as the ground truth, the joint angle errors for joints $P2$ and $P3$ of the numerical solver and the Simulink simulations were plotted against each other in Figure 7.

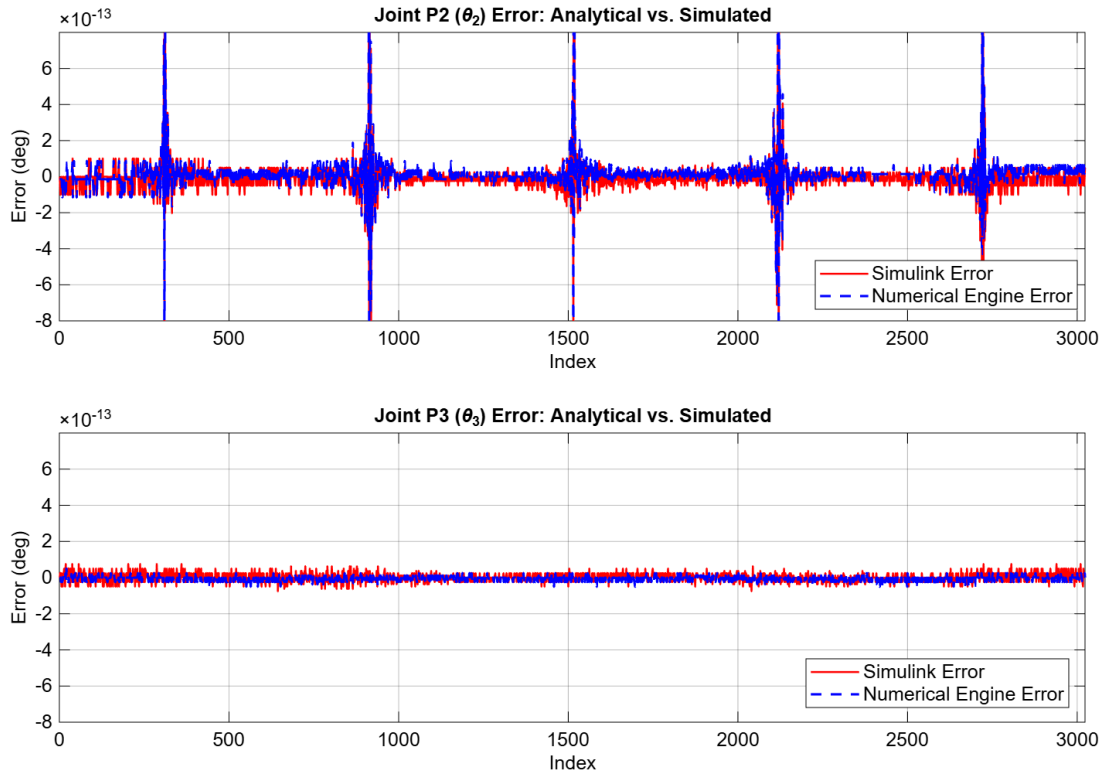


Figure 7: Numerical solver validation against analytical and Simulink model solutions for the four-bar mechanism.

These errors show how both the numerical and block diagram methods perform with respect to the ground truth. In all cases, the error for the custom solver is on the same order of magnitude as the error for the Simulink block diagram. Some points show greater errors for the $P2$ joint; these error spikes coincide perfectly with the mechanism reaching near-singular configurations, causing the Jacobian to become ill-conditioned. While greater in magnitude, these errors are still well within the 10^{-13} range which is acceptable. However in general, the errors lie consistently in the 10^{-14} range.

These results confirm that the custom parametric solver not only works, but that it achieves the same accuracy as commercial-grade solutions. By matching known analytical results, the solver establishes a robust foundation for evaluating higher complexity mechanisms, such as multi-loop systems, with no known analytical solutions.

5.2. Planar Scalability

While the four-bar linkage serves as an excellent baseline, the true robustness of the custom solver lies in its ability to handle mechanisms that are too complex for manual loop closure equation derivation. To verify the solver's ability to scale up in two-dimensional space, it was tasked with solving the Jansen leg mechanism [16].



Figure 8: Static posture verification of the Jansen linkage.

As shown in Figure 8, the custom solver successfully parsed the complex JSON architecture, allowing the automated graph-theory algorithm to generate the adjacency matrix and isolate the independent kinematic loops without any user intervention. The static posture was obtained with a maximum translational error of 2.29×10^{-16} m, proving that the automated loop discovery works on multi-loop planar mechanisms. In addition to static solutions, the solver is capable of providing users with a dynamic viewer. This dynamic viewer allows users to modify the input parameter(s) in real time to observe the mechanism's kinematic response. Figure 9 displays the resulting dynamic sweep of the input crank, where the motion of each joint in space can also be observed.

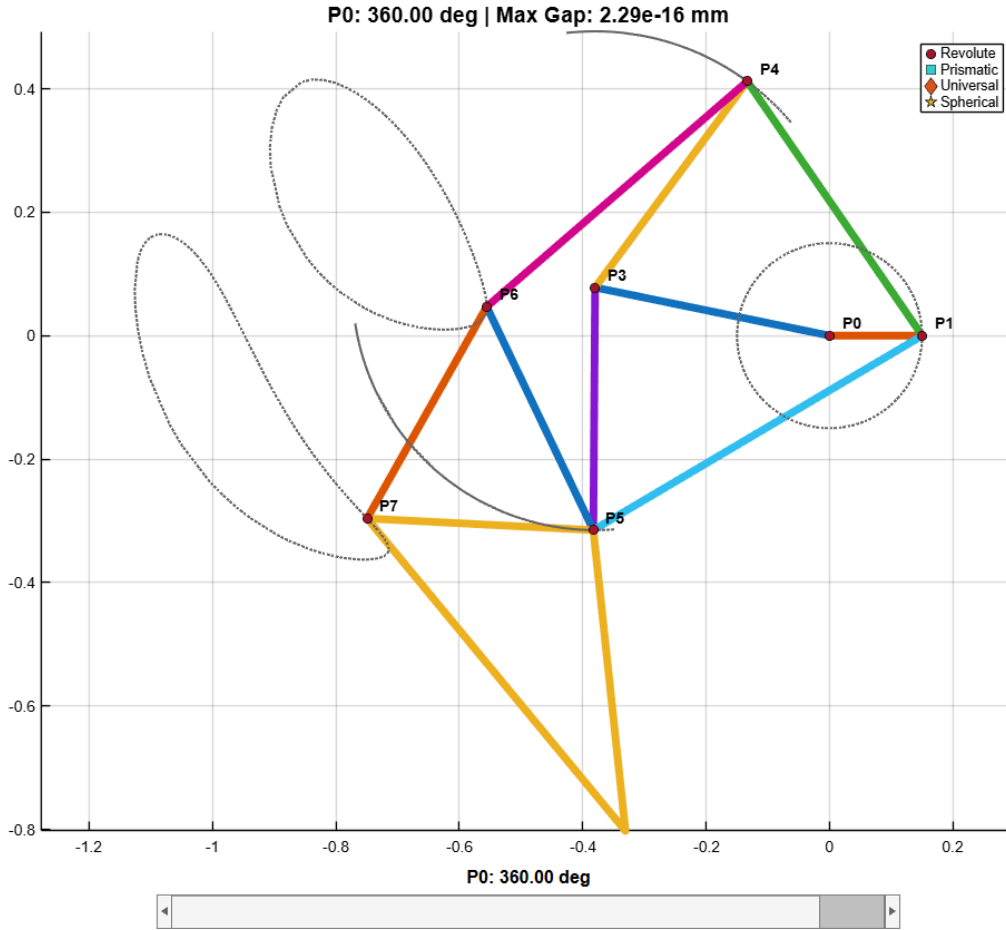


Figure 9: Dynamic posture verification of the Jansen linkage.

To further validate the solver’s scalability in two dimensions, tests were also performed on mechanisms such as: the Stephenson six-bar linkage, the Watt six-bar linkage, the Peaucellier-Lipkin mechanism, and others found in relevant literature [17, 18]. Across all these mechanisms, the loop discovery was executed successfully yielding high precision residuals, proving the flexibility of the planar workflow.

5.3. Spatial Validation

Having successfully proven that the solver is fully scalable in two dimensions, its ability to solve three dimensional kinematic chains was put to the test. To do this test, the solver was fed the spatial four-bar linkage [18]. This mechanism consists of two revolute joints and two spherical joints—a great way to test the spatial branch of the custom solver.

As shown in Figure 10, the solver successfully solved the posture of the spatial four-bar linkage with a maximum translation error of 1.92×10^{-12} m. This confirms that the principal axis-angle formulation employed in Section 4.4.2 correctly closes the kinematic loop. Ultimately, this successful convergence confirms that the custom solver is capable of handling complex spatial mechanisms with multiple degrees of freedom.

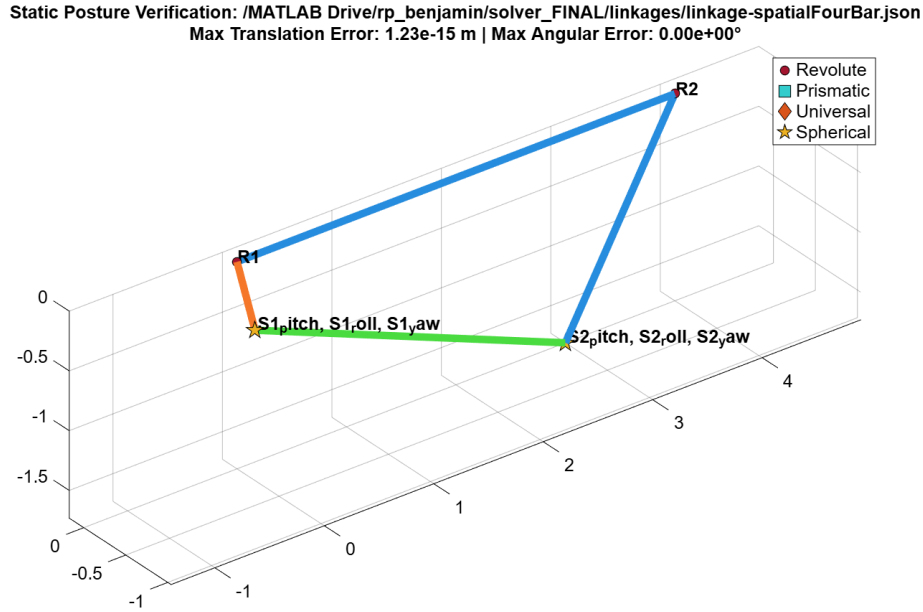


Figure 10: Static Posture verification of the Spatial Four-bar linkage.

5.4. Application to the ATHENA Hexapod

The development of the custom solver's architecture has led to this point-fulfilling the project's primary objective of automating the derivation of constrained parameters from free parameters for space robotics. To do this, the solver was tasked with solving the posture of the ESA Athena spacecraft's secondary mirror mechanism, the hexapod [4]. In the context of SDTlib, the hexapod's 6-DOF posture represents the free parameters while the six linear actuators, the hexapod's legs, represent the constrained parameters defined by nonlinear implicit equations.

5.4.1. Modelling the Hexapod

The hexapod was initialized in the JSON architecture using dimensions found in a published feasibility study. This study uses the following: a base radius of $r_b = 1.32$ m, a platform radius of $r_p = 1.20$ m, a base spacing angle of $\beta = 26^\circ$, and linear actuator lengths of $l = 0.685$ m [19].

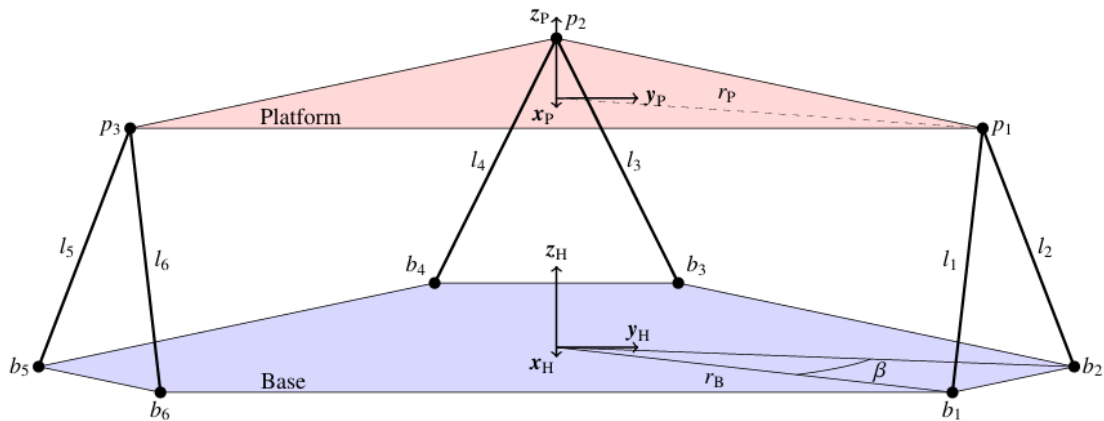


Figure 11: Sketch of the 6x3 hexapod mechanism (taken from [19])

An important detail to note with the Athena spacecraft's hexapod platform is that it has a 6x3 configuration. This means that the base has 6 independent connection points for the legs, while the platform only has 3 where pairs of legs share the same joints.

5.4.2. Resolving Overconstrained Singularities via Kinematic Equivalence

Due to the nature of the hexapod's topology and the desire to solve for the lengths of its linear actuators using a 6-DOF platform command, a virtual constraint had to be implemented. This was done with the help of a virtual body connecting the centre of the base to the centre of the platform using three prismatic joints and three perpendicular revolute joints (equivalent to a spherical joint). In theory, this made it possible to control the posture via the platform, but it inherently yielded six kinematic loops (one through each linear actuator). Because the custom solver uses the principal axis-angle formulation each loop has exactly six constraints (three translational and three rotational). This meant that with the virtual body, the hexapod resulted in a system of 36 equations.

However, because of the hexapod's 6x3 configuration, the total number of unknowns reduces to 27. Since three pairs of legs share a joint at the platform, the configuration in fact yields three kinematic loops each with nine unknowns. Therefore, the problem lies in the solver being incapable of converging due to the system being overconstrained as a result of having 36 constraint equations and 27 unknowns. Some of the constraint equations are redundant, but the solver does not have the functionality to automatically locate and remove them from the system of equations.

To address this limitation, the hexapod's topological definition was updated using an equivalent kinematic configuration. Instead of the leg pairs sharing the same joint at the platform, each leg was given their own independent joint, physically located at the same point as their pair. This successfully brought the number of unknown variables back up to 36, allowing the system of equations generated by the solver to be solved. While this updated topology makes it impossible to perfectly correlate the spherical joint angles with digital twins of the Athena's hexapod, the solution for the linear actuators remains a perfect match.

While this kinematic equivalence workaround was successful, the inability to natively resolve redundant constraints in overconstrained parallel platforms remains a limitation of the current architecture. Automating the identification and removal of these redundancies warrants future investigation.

5.4.3. Solving the Athena Hexapod

Having addressed the topology issue, the solver was tasked with solving the hexapod. As shown in Figure 12, the solver was able to find the posture of the hexapod with a maximum translation error of 2.62×10^{-16} m.

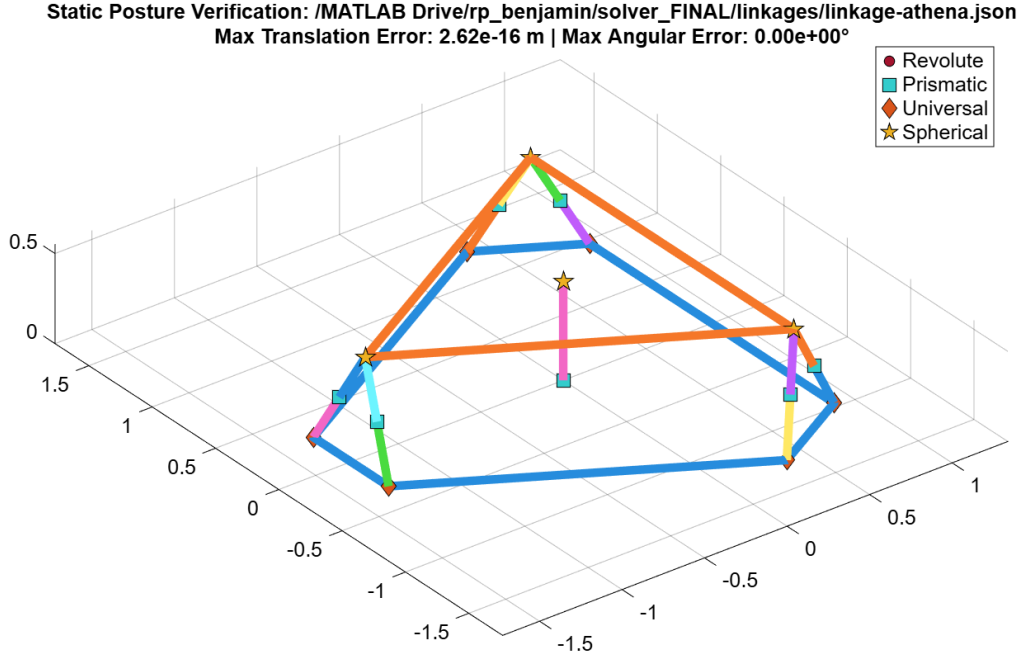


Figure 12: Static Posture verification of the Athena hexapod platform.

To prove that the solver obtained the correct constrained parameters, a verification using inverse kinematics was performed. The standard inverse kinematics vector loop closure equations for Stewart-Gough (hexapod) platforms is shown below in Eq. (12) [20]:

$$d_i = \|\mathbf{p} + {}^A R_B(\psi, \theta, \phi) {}^B \mathbf{b}_i - \mathbf{a}_i\| \quad (12)$$

Where d_i is the linear actuator length, $\mathbf{p}$ is the position vector of the platform, ${}^A R_B$ is the rotation matrix from the platform to the base, and ${}^B \mathbf{b}_i$ and $\mathbf{a}_i$ are the local platform and base attachment points. This was tested using the following commanded posture: $X = 0.2$ m, $Y = 0.3$ m, $Z = 0.1$ m, $\psi = 30^\circ$, $\theta = 15^\circ$, $\phi = 10^\circ$. Inputting this command in the custom solver and the analytical inverse kinematics equation found in Eq. (12) then makes it possible to do a results comparison of the linear actuator lengths as shown in Table 2.

Table 2: Analytical vs. Numerical Linear Actuator Lengths

Actuator	Analytical (m)	Numerical (m)	Error (m)
Leg 1	0.456	0.456	$\approx 10^{-15}$
Leg 2	0.096	0.096	$\approx 10^{-15}$
Leg 3	0.589	0.589	$\approx 10^{-15}$
Leg 4	0.444	0.444	$\approx 10^{-15}$
Leg 5	0.638	0.638	$\approx 10^{-15}$
Leg 6	0.140	0.140	$\approx 10^{-15}$

This shows that the custom solver yields results that are almost identical to those found analytically. Ultimately, this validates the fact that the custom solver is capable of computing the constrained parameter values for systems with the level of complexity of the hexapod platform.

6. Conclusion

The primary objective of this research was to develop a generalized parametric numerical solver capable of computing the constrained posture of closed kinematic chains. By breaking down mechanisms into a standardized JSON format and using graph theory for automated loop discovery, the custom solver successfully replaced the demanding process of manually deriving explicit loop closure equations—a process necessary for approximating LFTs for the implementation of closed loop mechanisms in SDTlib.

The primary contribution of this research lies in the solver’s automated kinematic loop discovery and generalized mathematical formulation. Rather than relying on hardcoded algebraic constraints or computationally heavy commercial software, the developed architecture uses graph theory to autonomously map the topology of any closed kinematic chain. The use of the principal axis-angle formulation, in addition to the Levenberg-Marquardt optimization algorithm resulted in a robust parametric posture solver for closed kinematic chains. This was proven through the solver’s ability to evaluate highly complex mechanisms such as the ESA Athena spacecraft’s hexapod platform. A verification of the results using established analytical inverse kinematics showed a solver accuracy of as low as 10^{-16} m.

While the solver is highly robust, certain limitations leave room for future development. As shown with the Athena hexapod, kinematic equivalence needed to be employed in the JSON definition to make the system solvable. While mathematically correct, it prevents the solution from being a true 1-to-1 digital recreation of the Athena hexapod. As such, future work should focus on implementing a functionality capable of identifying redundant constraints.

Additionally, future developments should aim to reduce the verbosity of the JSON mechanism definitions. For simple mechanisms like the four-bar linkage they are manageable, but for complex mechanisms like the Athena hexapod, modelling multi-DOF connections using primitive 1-DOF joints results in lengthy JSON files. This could be streamlined by implementing specific code that can handle joints like universal and spherical joints. It would significantly reduce the complexity of the JSON definitions and be a great quality of life improvement for users.

References

- [1] Cornelius Dennehy and Oscar S. Alvarez-Salazar. *Spacecraft Micro-Vibration: A Survey of Problems, Experiences, Potential Solutions, and Some Lessons Learned*. Tech. rep. NASA, 2018. URL: <https://ntrs.nasa.gov/api/citations/20180006315/downloads/20180006315.pdf>.
- [2] Daniel Alazard and Francesco Sanfedino. *Satellite Dynamics Toolbox library (SDTlib) - User's guide*. Tech. rep. Institut Supérieur de l'Aéronautique et de l'Espace, 2021.
- [3] Kemin Zhou and John Comstock Doyle. *Essentials of Robust Control*. Vol. 104. Upper Saddle River, NJ: Prentice Hall, 1999. ISBN: 9780137739790.
- [4] Simon Görries. "ATHENA Space Telescope: Line of Sight Control with a Hexapod in the Loop". MA thesis. Stockholm, Sweden: KTH Royal Institute of Technology, 2017.
- [5] Tomomichi Sugihara. "Solvability-unconcerned inverse kinematics by the Levenberg–Marquardt method". In: *IEEE Transactions on Robotics* 27.5 (2011), pp. 984–991.
- [6] Jens Wittenburg. *Dynamics of Multibody Systems*. 2nd. Springer Science & Business Media, 2008.
- [7] Werner Schiehlen. "Multibody system dynamics: roots and perspectives". In: *Multibody System Dynamics* 1.2 (1997), pp. 149–188.
- [8] Alessandro Tasora et al. "Chrono: An open source multi-physics dynamics engine". In: *High Performance Computing in Science and Engineering*. Springer, 2016, pp. 19–49.
- [9] Taylor Howell et al. "Dojo: A differentiable physics engine for robotics". In: *arXiv preprint arXiv:2203.00806* (2022).
- [10] Lung-Wen Tsai. *Mechanism design: enumeration of kinematic structures according to function*. CRC press, 2000.
- [11] Andreas Müller. "Representation of the kinematic topology of mechanisms for kinematic analysis". In: *Mechanism and Machine Theory* 90 (2015), pp. 88–105.
- [12] Jorge Nocedal and Stephen J Wright. *Numerical optimization*. Springer, 2006.
- [13] James Diebel. *Representing attitude: Euler angles, unit quaternions, and rotation vectors*. Tech. rep. Stanford University, 2006.
- [14] Danail Brezov. "Optimization and Gimbal Lock Control via Shifted Decomposition of Rotations". In: *Journal of Applied & Computational Mathematics* 7.4 (2018).
- [15] The MathWorks, Inc. *cyclebasis: Fundamental cycle basis of graph*. MATLAB Documentation. The MathWorks, Inc. Natick, Massachusetts, United States, 2026. URL: <https://www.mathworks.com/help/matlab/ref/graph.cyclebasis.html>.
- [16] Lalit Patnaik and Loganathan Umanand. "Kinematics and dynamics of Jansen leg mechanism: A bond graph approach". In: *Simulation Modelling Practice and Theory* 60 (2016), pp. 160–169. DOI: 10.1016/j.simpat.2015.10.003.
- [17] Joseph Edward Shigley and John Joseph Uicker. *Theory of Machines and Mechanisms*. Fifth. Oxford University Press, 1980. ISBN: 9781009303675. DOI: <https://doi.org/10.1017/9781009303644>.
- [18] John J. Uicker, Gordon R. Pennock, and Joseph E. Shigley. *Theory of Machines and Mechanisms*. 5th ed. New York: Oxford University Press, 2017. ISBN: 978-0-19-026448-2.

- [19] Alison Ponche et al. “Derivation and Verification of a Flexible Hexapod Model for Multi-Body Space Observatories”. In: *Aerospace Europe Conference 2023 - 10th EUCASS - 9th CEAS*. 2023. DOI: 10.13009/EUCASS2023-374.
- [20] Lung-Wen Tsai. *Robot Analysis: The Mechanics of Serial and Parallel Manipulators*. New York: John Wiley & Sons, 1999. ISBN: 978-0-471-32593-2.

A. Four-bar Mechanism JSON File Example

```

1 {
2   "bodies": [
3     {
4       "name": "Frame",
5       "points": [
6         {"id": "P0", "location": [0, 0, 0]},
7         {"id": "P1", "location": [-0.254, 0, 0]}
8       ]
9     },
10    {
11      "name": "Crank",
12      "points": [
13        {"id": "P1", "location": [-0.053975, 0, 0]},
14        {"id": "P2", "location": [0.053975, 0, 0]}
15      ]
16    },
17    {
18      "name": "Follower",
19      "points": [
20        {"id": "P2", "location": [-0.1397, 0, 0]},
21        {"id": "P3", "location": [0.1397, 0, 0]}
22      ]
23    },
24    {
25      "name": "Lever",
26      "points": [
27        {"id": "P3", "location": [-0.135255, 0, 0]},
28        {"id": "P0", "location": [0.135255, 0, 0]}
29      ]
30    }
31  ],
32  "joints": [
33    {
34      "label": "P0",
35      "bodies": ["Frame", "Lever"],
36      "type": "revolute",
37      "axis": [0, 0, 1]
38    },
39    {
40      "label": "P1",
41      "bodies": ["Frame", "Crank"],
42      "type": "revolute",
43      "axis": [0, 0, 1]
44    },
45    {
46      "label": "P2",
47      "bodies": ["Crank", "Follower"],
48      "type": "revolute",
49

```

```
50     "axis": [0, 0, 1]
51   },
52   {
53     "label": "P3",
54     "bodies": ["Follower", "Lever"],
55     "type": "revolute",
56     "axis": [0, 0, 1]
57   }
58 ]
59 }
```

B. JSON File Nomenclature

To ensure the custom solver can process complex spatial geometries, the JSON architecture requires specific formatting and nomenclature for different elements. This section provides the necessary syntax to define ternary linkages, prismatic actuators, and spherical joints. These definitions are sufficient to develop most bodies and joint types.

B.1. Ternary Links

The solver supports bodies with any number of connections. For example, a ternary link can be defined by adding a third point object to the body's array. The adjacency matrix will automatically recognize all branches.

```

1 {
2   "name": "Ternary_Link",
3   "points": [
4     {"id": "Joint_A", "location": [0.0, 0.0, 0.0]},
5     {"id": "Joint_B", "location": [0.5, 0.5, 0.0]},
6     {"id": "Joint_C", "location": [1.0, 0.0, 0.0]}
7   ]
8 }
```

B.2. Prismatic Joints

By default, the solver assumes joints are revolute. To define a linear actuator, the **type** must explicitly be set to **prismatic**. Furthermore, the **axis** array must contain the normalized 3D unit vector defining the exact line of action along which the actuator slides.

```

1 {
2   "label": "Linear_Actuator",
3   "bodies": ["Lower_Leg", "Upper_Leg"],
4   "type": "prismatic",
5   "axis": [0.707, 0.707, 0.0]
6 }
```

B.3. Spherical Joints

Because the solver parses topologies using fundamental 1-DOF revolute joints, a 3-DOF spherical joint must be designed using kinematic equivalence. This is achieved by defining three intersecting revolute joints connected at the exact same Cartesian location.

The following example demonstrates a spherical joint connecting a a leg to a platform using a Yaw-Pitch-Roll sequence:

```

1 "bodies": [
2   {
3     "name": "Spherical_1",
4     "points": [
5       {"id": "S_yaw", "location": [0.5, 0.5, 1.0]},
6       {"id": "S_pitch", "location": [0.5, 0.5, 1.0]}

```

```
7     ]
8   },
9   {
10    "name": "Spherical_2",
11    "points": [
12      {"id": "S_pitch", "location": [0.5, 0.5, 1.0]},
13      {"id": "S_roll", "location": [0.5, 0.5, 1.0]}
14    ]
15  }
16 ],
17 "joints": [
18   {
19     "label": "S_yaw",
20     "bodies": ["Leg", "Spherical_1"],
21     "type": "revolute",
22     "axis": [0, 0, 1]
23   },
24   {
25     "label": "S_pitch",
26     "bodies": ["Spherical_1", "Spherical_2"],
27     "type": "revolute",
28     "axis": [1, 0, 0]
29   },
30   {
31     "label": "S_roll",
32     "bodies": ["Spherical_2", "Platform"],
33     "type": "revolute",
34     "axis": [0, 1, 0]
35   }
36 ]
```